What to record for reproducible experiments

Minimum recording checklist

The COGANT pipeline is deterministic on a fixed filesystem snapshot under a fixed Python interpreter, so bit-for-bit reproduction of a published run reduces to pinning exactly the inputs that the pipeline consumes and the environment it consumes them in. For every experiment whose output you intend to re-run, redistribute, or cite, record the following:

1. **COGANT version and commit hash.** The `__version__` string in `../cogant/py/cogant/__init__.py` matches `project.version` in `../cogant/pyproject.toml` (currently `0.5.0`). Also capture the Git SHA of the checkout used — `git rev-parse HEAD` from inside the `cogant/` subtree — because a single version tag can span several bug-fix commits and the reproducibility contract is at commit granularity, not tag granularity. The shipped `cogant doctor` command prints both values.
2. **Python interpreter version.** Either `CPython 3.11.x`, `3.12.x`, or `3.13.x` per the `classifiers` block in `pyproject.toml`; the canonical benchmark run used

How to re-run the canonical fixture experiments

All numbers in Tables 4–7 of §6.3 and in the benchmark suite of §6.4 are regenerated by two scripts that live inside the COGANT package tree and take no arguments beyond a `--output` directory:

```
# One-shot regeneration of every figure and the metrics.js
uv run python ../cogant/evaluation/figures/generate_figures
    --output ../cogant/evaluation/figures/

# Benchmark harness - three iterations per fixture, stage-
uv run python ../cogant/benchmarks/bench_suite.py \
    --iterations 3 \
    --output ../cogant/benchmarks/results/
```

The figure-generation script re-runs the `RoundtripOrchestrator` (`../cogant/examples/orchestrate_roundtrip.py`) against every fixture under `../cogant/examples/control_positive/` and `../cogant/examples/real_world/`, re-reads the emitted JSON, writes `metrics.json` alongside `fig1_graph_sizes.png`, `fig2_node_kinds.png`, `fig3_state_space.png`, and `fig4_pipeline_latency.png`, and finally copies the canonical

What the manifest.json index records

Every `gnn_package/` directory emitted by the pipeline includes a `manifest.json` file whose job is to close the reproducibility loop without requiring the downstream consumer to re-hash the bundle. The manifest records: the COGANT version that produced the bundle, the interpreter and platform identifiers from step 2 of the checklist, the list of stages actually executed (step 6), the schema name passed to the state-space compiler, the paths of every file in the `gnn_package/` directory, and the SHA-256 hash of each file. A reproducer can therefore verify a published bundle with a single pass over the manifest, without consulting any external metadata file.

Worked example: regenerating the Table 4 row for `flask_app`

The canonical Table 4 row for the `flask_app` fixture (six files, 853 lines, 98 nodes, 597 edges, 14.58 s wall-clock) was produced as follows, and the command sequence below should bit-for-bit reproduce the same `program_graph.json` and `state_space.json` on any macOS arm64 machine with the same uv lockfile:

```
cd projects_in_progress/cogant/cogant                    # package r
uv sync --extra all                                        # pins ev
git rev-parse HEAD                                         # record
python --version && uname -a                               # record
uv run cogant translate \
    examples/real_world/flask_app \
    --output /tmp/repro_flask_app \
    --layout-output
uv run cogant validate /tmp/repro_flask_app/gnn_package
sha256sum /tmp/repro_flask_app/gnn_package/*.json | sort
```

Data ethics and licensing for exported bundles

Every COGANT bundle can contain identifiers, docstrings, and inline comments lifted verbatim from the input source code. When redistributing a bundle derived from a third-party repository — for example when publishing a dataset of `gnn_package/` outputs from open-source Python projects — the downstream license terms must be respected: the original repository's license applies to any prose or identifier that the bundle quotes, and the COGANT MIT license covers only the pipeline code and the synthetic fixtures it ships. Organisations with private data policies should additionally review bundles for personally identifiable information before publication; the pipeline does not perform PII scrubbing on its own. The short policy statement is in §7 (see **Data ethics and licensing** in `07_reproducibility.md`); this subsection supplies the actionable hashing and manifest-recording recipe.

Cross-references

- The CLI hub at `../cogant/docs/cli/README.md` links to every flag that changes the recorded-output shape, and the stage list in `../cogant/docs/architecture/README.md` enumerates the ten-stage DAG.
- The per-release narrative in `../cogant/CHANGELOG.md` and `../cogant/RELEASE_NOTES.md` documents which default-behaviour changes (for example the v0.5.0 POLICY / CONTEXT stub-emission fix discussed in §5 and `ROUNDTRIP_IMPROVEMENT.md`) could affect a re-run against a pre-v0.5.0 bundle.
- The calibration sweep plan in `../cogant/docs/evaluation/CALIBRATION.md` is the canonical reference for the `TODO(calibration)` markers cited in §5; re-running a confidence-threshold sweep requires the 20+ repository gold-standard corpus discussed there.